1 Question 1

1.1 Objective

Send one unit from S to T and minimize the total shipping cost.

$$\min\ 8x_{0,1} + 6x_{0,4} + x_{1,2} + 3x_{1,3} + x_{2,5} + 5x_{3,4} + 3x_{3,5} + 5x_{4,5}$$

st

$$0 \le x_{0,1} \le 5 \tag{1}$$
$$0 \le x_{0,4} \le 1 \tag{2}$$
$$0 \le x_{1,2} \le 1 \tag{3}$$
$$0 \le x_{1,3} \le 2 \tag{4}$$
$$0 \le x_{2,5} \le 1 \tag{5}$$
$$0 \le x_{3,4} \le 2 \tag{6}$$
$$0 \le x_{3,5} \le 3 \tag{7}$$
$$0 \le x_{4,5} \le 2 \tag{8}$$
$$x_e \ge 0 \quad \forall\, e \in E \tag{9}$$

1.2 Code

```
sol=linprog(costs, A_eq=A, b_eq=ballance, bounds=bounds, method="highs")
```

Where:
costs = **cost of each edge** (e)**.**
A = **the directional adjacency matrix**
ballance = **ending and starting amount**
bounds = **capacity of each edge**

1.3 Solution

$$Optimalcost : 10.0 \tag{10}$$
$$path : (0,1) \rightarrow (1,2) \rightarrow (2,5) \tag{11}$$

2 Question 2

2.1 Objective

Minimize the cost of purchasing laptops for UNM

$$min 600a + 2300b$$

st

$$-a - b \leq -10 \tag{12}$$
$$600a + 2300b \leq 10,000 \tag{13}$$
$$a - 2d \leq 0 \tag{14}$$
$$-2a + d \leq 0 \tag{15}$$

2.2 code

```
sol1 = linprog(c, A_eq = A1, b_eq = b1, bounds = bounds, method = "highs")
```

Where:
$c =$ **cost per unit [600, 2300]**
$A =$ **directional adjacency matrix.**
$b =$ **constraints [-10, 10,000, 0, 0]**

2.3 Solution

Solution: None.
The problem is impossible to answer because it is impossible to obtain the necessary laptops given the constraints.

I ran my program with a 20,000 dollar limit and here is the cheapest price for 10 laptops following all other constraints
Solution: 11666.666666666666

3 Question 3

3.1 Part A

Little o means that there is a strong upper bound when comparing the run-time.

That is: $f(n) = o(g(n)) \rightarrow limf(n)/g(n) = 0$

$f(n)$ **becomes infinitely small as n becomes arbitrarily large**

To achieve o(n), in the case of $g(n) = n$**, f(n) is a continuous function :** $f(n) = log(n)$**,** $f(n) = n^{1/2}$**, or any continuous function that increases slower than** n

3.2 Part B

To find a card x **where** $x_{i-1} \geq x_i \leq x_{i+1}$ **I will implement a varied binary search.**
I will start by flipping $x_i, wherei = n/2$
I will then, if possible, flip its neighbors.
I will repeat this process on either half of this midpoint until I find a local minimum or n cards have been flipped

3.3 code

```
def find_local_min(arr, low, high):
"""
binary search without sorting
"""
if low == high:
    return arr[low]

i = (low + high) // 2

left_ok  = (i == 0) or (arr[i] <= arr[i - 1])
right_ok = (i == len(arr) - 1) or (arr[i] <= arr[i + 1])

if left_ok and right_ok:
    return arr[i]

if i > 0 and arr[i - 1] < arr[i]:
    return find_local_min(arr, low, i - 1)
return find_local_min(arr, i + 1, high)
```

4 Question 4

To solve the shortest path using linear programming, assume that all edges have equal inflow and outflow except node
$S \rightarrow +1$ **,** $T \rightarrow -1$**,**
Then find the shortest path distance from $S \rightarrow T$

5 Question 5

5.1 Bellmond Ford

After n iterations the algorithm has found the shortest path using at most n edges from the source to every vertex.
Shortest Path: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7$

5.2 Dikstra's

In each iteration the shortest path calculated from the source node source to any vertex is the shortest path and will not change
Shortest Path: $0 \rightarrow 6 \rightarrow 7$

5.3 Floyd Warshall

let k be the number of intermediate vertices. The algorithm finds the shortest path between each vertex using at most k vertices. $k = 0, 1, 2...n-1$ **, where n is the number of nodes**
Shortest Path: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7$

6 Question 6

6.1 Introduction

I have chosen to represent my graph as an agency list. To test if an undirected graph is a tree we must determine that:

- the graph has no cycles
- that every vertex can be visited from the starting vertex.

6.2 Test Trees and Results

```
tree_graph = {
    0: [1],
    1: [0, 2],
    2: [1]
}

not_tree_graph = {
    0: [1, 2],
    1: [0, 2],
    2: [0, 1]
}
print(is_tree(3, tree_graph))
print(is_tree(3, not_tree_graph))

Testing tree_graph:  True
Testing not_tree_graph:  False
```

It should be mentioned that I also check if the graph has 0 or 1 node since, if that is the case, we can return true right away... efficiency

algorithms on next page

6.3 Algorithm

```
    def is_tree(num, dict):
    """
    checks if given graph is a tree
    """
    if (num == 0):
        return True

    visited = [False] * num
    parent = [-1] * num

    if not check_no_cycles(0, visited, parent, dict):
        return False

    for i in range(num):
        if not visited[i]:
            return False

    edges = 0
    for i in range(num):
        edges += len(dict.get(i, []))
    edges //= 2
    if not edges == num - 1:
        return False

    return True


def check_no_cycles(u, visited, parent, dict):
    """
    Used to detect cycles
    """
    visited[u] = True
    for v in dict.get(u, []):
        if not visited[v]:
            parent[v] = u
            if not check_no_cycles(v, visited, parent, dict):
                return False
        elif v != parent[u]:
            return False
    return True
```

7 Question 7

7.1 Objective

Send n families on a road trip with each family, member from a given family, in a separate car

7.2 Constraints

- Each family member must be in a separate car
- Cars must have enough seats to support at most one member from each family

7.3 Network Flow

I will first construct a graph. The source will lead to n nodes, n is the number of families. The edges between the source and these family nodes will have a capacity equal to the number of family members. These family nodes will each have edges pointing to c car nodes, c is the number of cars. The number of edges coming from the family nodes will equal the number of family members and each have capacity one. no two nodes from a single family node will point to the same car node.
The car nodes will each have an edge pointing to the sink, t. The capacity on each of these edges will equal the number of seats in each car.

7.4 Conclusion

If all goes well the flow, from source, will match the flow into the sink. Each family member will be in a different car and everyone will be happy. There is no way that two members of the same family can be in the same car, and the only problem would be that there are cars with too few seats to support the solution.